Design e Arquitectura de Software

2024/2025

Lab. Classes - 9

Java web services with Spring Boot

The objective of this lesson will be to implement the long running operation and re-runnable operation patterns.

1 – Start by creating a class/resource Search that holds a sequence of strings. These represent the search strings entered in a search engine during a given period. These are added with the :addSearchString custom method, e.g.:

```
/searches/id=12-06-2022:add?searchString=”fifa”
```

2- Implement the custom method mostCommon. This returns the most commonly searched topic. E.g:

```
/searches:mostCommon
```

Because this is an operation that might potentially take long to complete, it should use the long running operation pattern. The LRO should run (do its work) in a new thread*, while the LRO object is returned immediately.
The LRO can be accessed as any other resource and have the usual fields (check the theoretical slides).

(to be able to test your code, you can introduce an explicit delay (e.g. 20 seconds) in order to make it easier to interact with the LRO.)
** Alternatively, for the purposes of this example, you can create a custom method that you can call to “artificially” end the LRO computation. E.g. after calling /searches:mostCommon the LRO will not complete until /searches:completeOperation is called).*

3 – You want to provide the ability to periodically sweep the searched keywords so that a few selected search strings deemed to be considered inappropriate are removed from the server records. The list of search strings to remove is configurable, but changes seldom, and it can be quite extensive. Rather than explicitly providing this list each time the purge method is called, i.e.:

```
/searches:purge?key=”profanity1”&key=”profanity2”& …<etc>
```

Which might be cumbersome and time consuming if the list is long, we will create a re-runnable job with the appropriate configuration, which will then be run at a subsequent time. E.g. It can be created :

```
POST /jobs
{
“keyList”:  ["profanity1", "profanity2", "profanity3",…]
}
```

And then executed at a later date using the run custom method. It should return an LRO that allows the user to monitor execution.

```
POST /jobs/1:run
```

The result of execution should be accessed via the top-level `executions` resource.